

Point Transformer

Hengshuang Zhao^{1,2} Li Jiang³ Jiaya Jia³ Philip Torr¹ Vladlen Koltun⁴

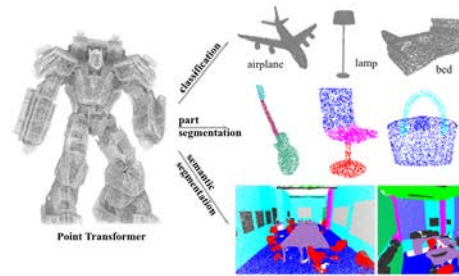
¹University of Oxford ²The University of Hong Kong

³The Chinese University of Hong Kong ⁴Intel Labs

CV 26 Sep 2021

Abstract

Self-attention networks have revolutionized natural language processing and are making impressive strides in image analysis tasks such as image classification and object detection. Inspired by this success, we investigate the application of self-attention networks to 3D point cloud processing. We design self-attention layers for point clouds and use these to construct self-attention networks for tasks such as semantic scene segmentation, object part segmentation, and object classification. Our Point Transformer design improves upon prior work across domains and tasks. For ex-



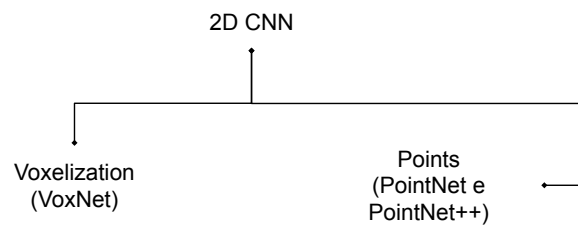
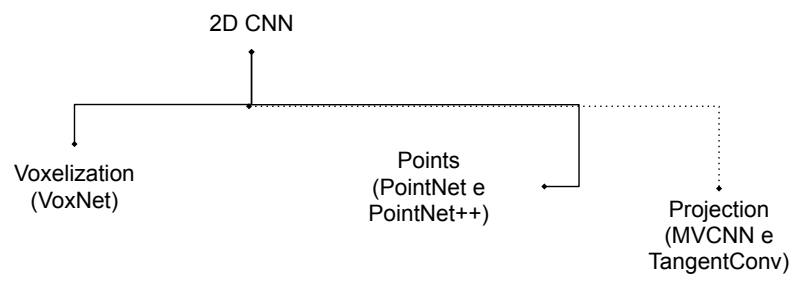
Object

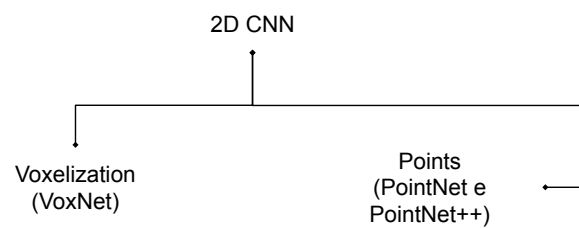
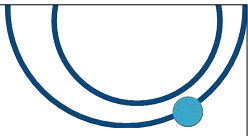
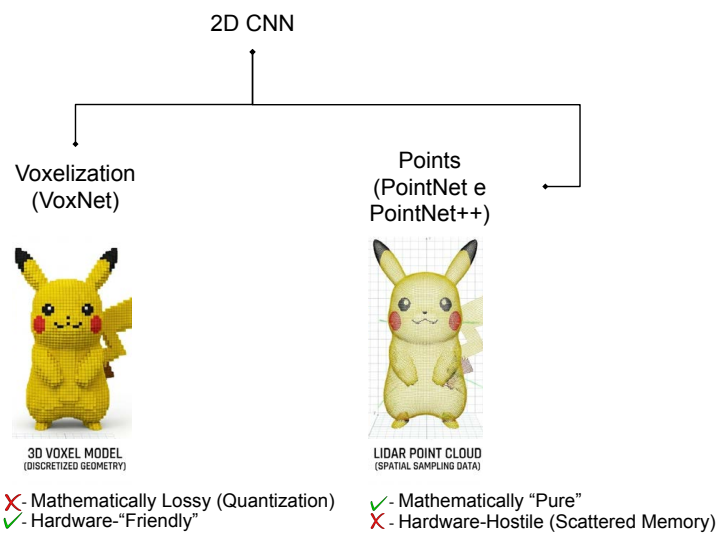
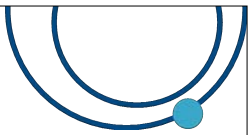


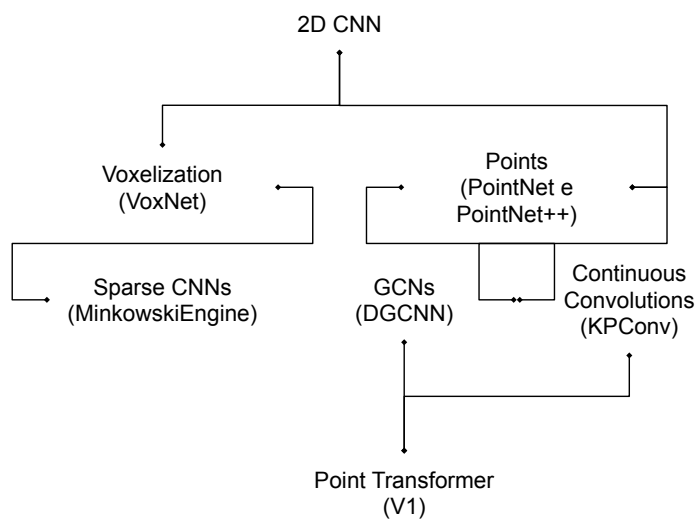
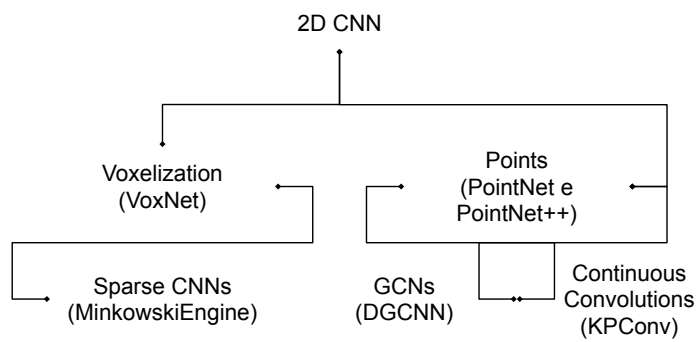
Points

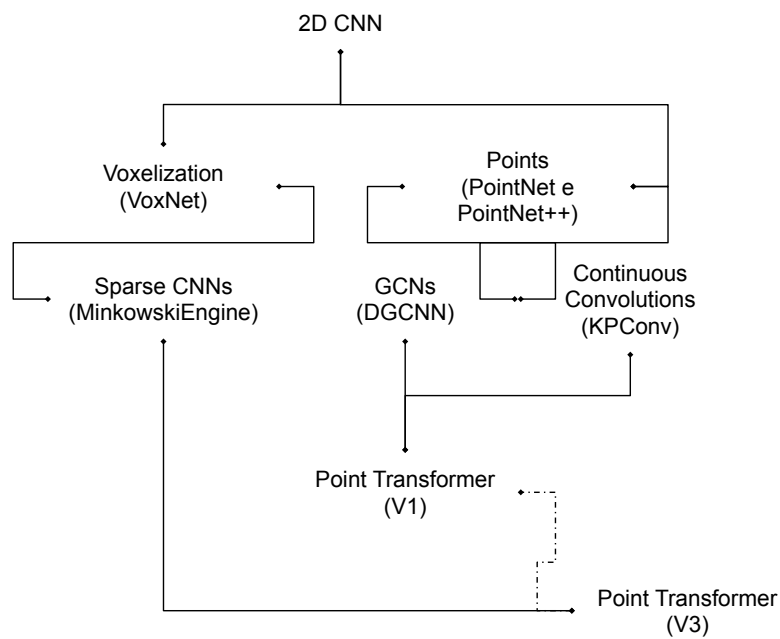


Voxels

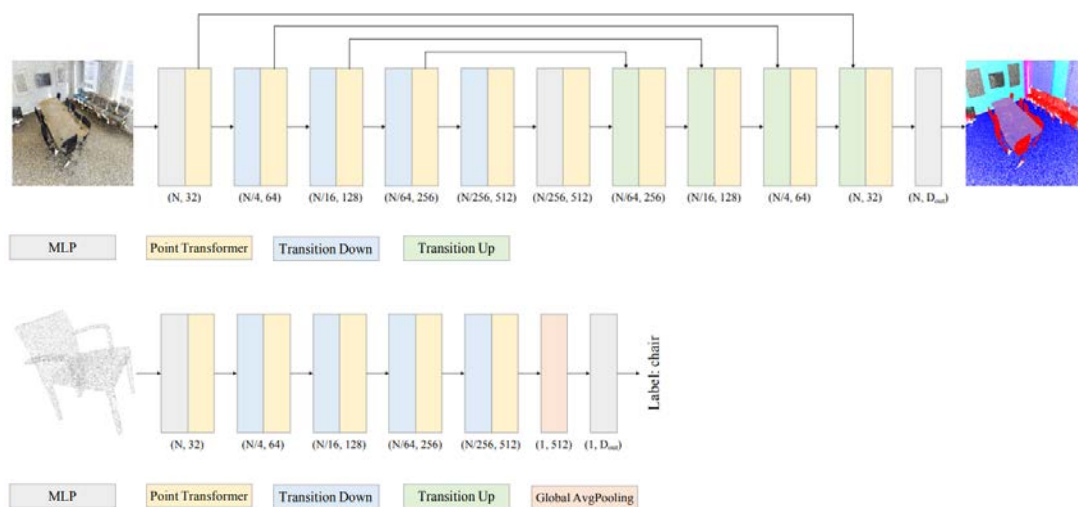




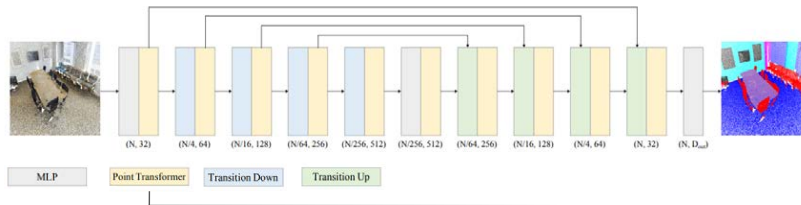




Architecture and Key Aspects

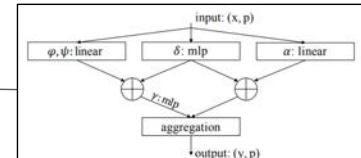


Architecture and Key Aspects



Step-by-Step Architecture Walkthrough:

- **Step 1: Local Graph Construction (K-NN):** The model takes the raw point cloud. Because there is no grid, it performs a K-Nearest Neighbors (K-NN) search around every point to establish a localized tracking region.
- **Step 2: Linear Projection:** The features of the center point (i) and neighboring points (j) are projected into Query, Key, and Value spaces via shared Linear layers.
- **Step 3: Relative Position Embedding (δ):** Simultaneously, the model subtracts the raw coordinates ($p_i - p_j$) and passes the difference through a small MLP. This produces δ , a continuous spatial encoding vector.
- **Step 4: Vector Weight Generation:** Instead of doing a standard dot-product, the Query vector and Key vector are *subtracted*, added to the spatial vector δ , and passed through a mapping network (γ). This creates a unique attention weight for *every single feature channel*.
- **Step 5: Element-Wise Modulation & Aggregation:** These vector weights are normalized via Softmax and multiplied element-wise ($\odot$) with the Value vector. The results within the k-NN neighborhood are summed up to output the new point feature.



Implementation Details – Modules

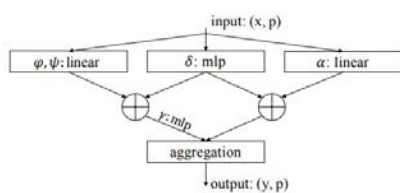


Figure 2. Point transformer layer.

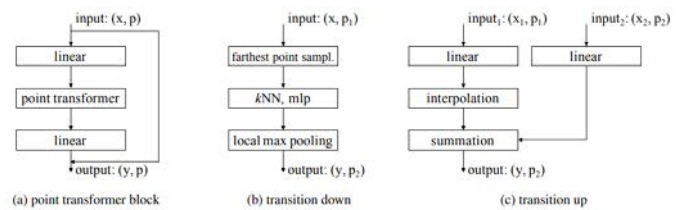


Figure 4. Detailed structure design for each module.

Chunghyun Park, 4 years ago | 1 author (Chunghyun Park)
> `class PointTransformerLayer(nn.Module): ...`

Chunghyun Park, 4 years ago | 1 author (Chunghyun Park)
> `class TransitionDown(nn.Module):`

Chunghyun Park, 4 years ago | 1 author (Chunghyun Park)
> `class TransitionUp(nn.Module): ...`

Chunghyun Park, 4 years ago | 1 author (Chunghyun Park)
> `class PointTransformerBlock(nn.Module): ...`

Implementation Details – Inside a Module

```
class PointTransformerLayer(nn.Module):
    def __init__(self, in_planes, out_planes, share_planes=8, nsample=16):
        super().__init__()
        self.mid_planes = mid_planes = out_planes // 1
        self.out_planes = out_planes
        self.share_planes = share_planes
        self.nsample = nsample
        self.linear_q = nn.Linear(in_planes, mid_planes)
        self.linear_k = nn.Linear(in_planes, mid_planes)
        self.linear_v = nn.Linear(in_planes, out_planes)
        self.linear_p = nn.Sequential(nn.Linear(3, 3), nn.BatchNorm1d(3), nn.ReLU(inplace=True), nn.Linear(3, out_planes))
        self.linear_w = nn.Sequential(nn.BatchNorm1d(mid_planes), nn.ReLU(inplace=True),
                                      nn.Linear(mid_planes, mid_planes // share_planes),
                                      nn.BatchNorm1d(mid_planes // share_planes), nn.ReLU(inplace=True),
                                      nn.Linear(out_planes // share_planes, out_planes // share_planes))
        self.softmax = nn.Softmax(dim=1)

    def forward(self, pxo) -> torch.Tensor:
        p, x, o = pxo # (n, 3), (n, c), (b)
        x_q, x_k, x_v = self.linear_q(x), self.linear_k(x), self.linear_v(x) # (n, c)
        x_k = pointops.queryandgroup(self.nsample, p, p, x_k, None, o, o, use_xyz=True) # (n, nsample, 3+c)
        x_v = pointops.queryandgroup(self.nsample, p, p, x_v, None, o, o, use_xyz=False) # (n, nsample, c)
        p_r, x_k = x_k[:, :, 0:3], x_k[:, :, 3:]
        for i, layer in enumerate(self.linear_p):
            p_r = layer(p_r.transpose(1, 2).contiguous()).transpose(1, 2).contiguous() if i == 1 else layer(p_r) # (n, nsample, c)
        w = x_k - x_q.unsqueeze(1) + p_r.view(p_r.shape[0], p_r.shape[1], self.out_planes // self.mid_planes, self.mid_planes).sum(2) # (n, nsample, c)
        for i, layer in enumerate(self.linear_w):
            w = layer(w.transpose(1, 2).contiguous()).transpose(1, 2).contiguous() if i % 3 == 0 else layer(w)
        w = self.softmax(w) # (n, nsample, c)
        n, nsample, c = x_v.shape; s = self.share_planes
        x = ((x_v + p_r).view(n, nsample, s, c // s) * w.unsqueeze(2)).sum(1).view(n, c)
        return x
```

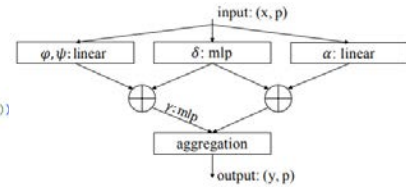
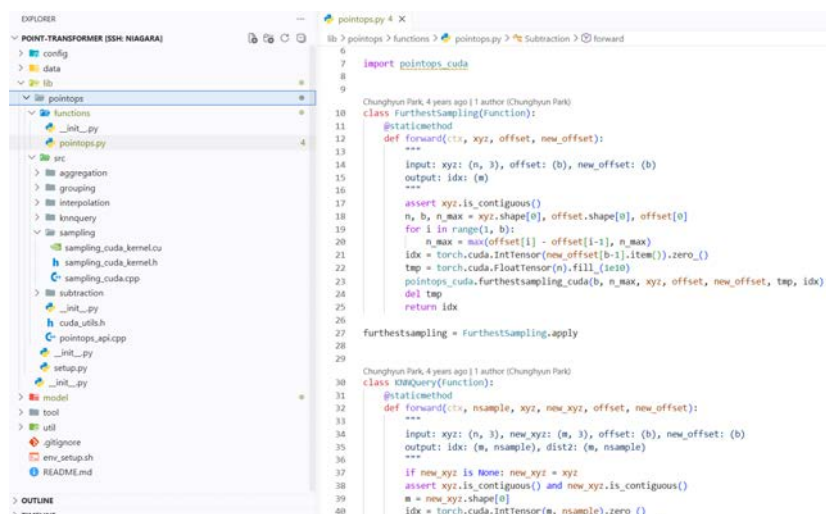


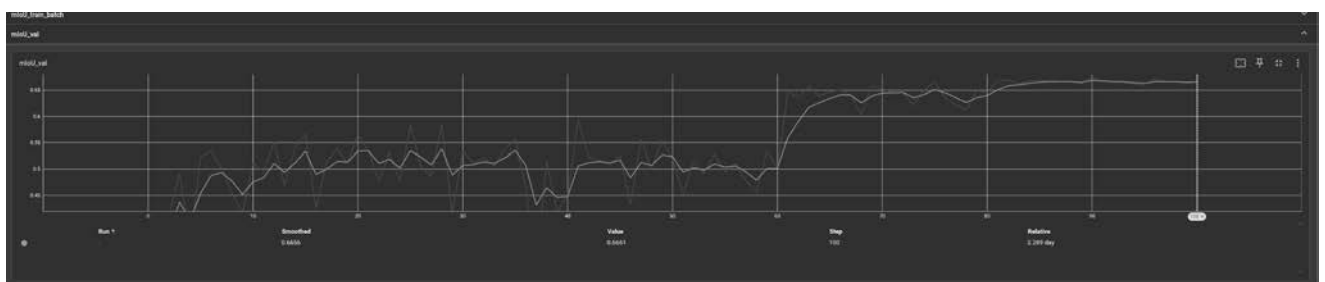
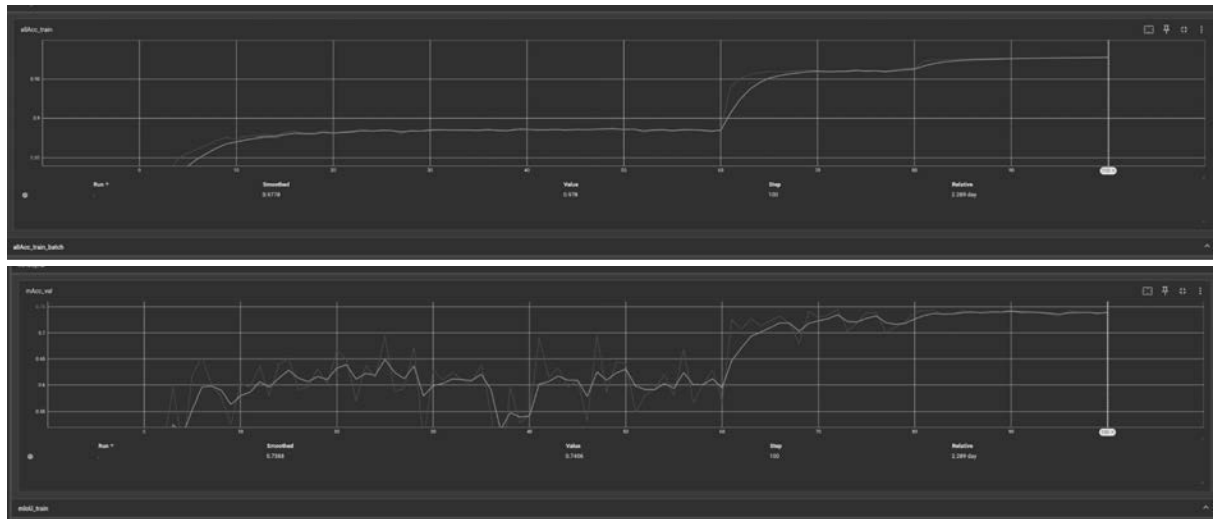
Figure 2. Point transformer layer.

Implementation Details – pointops Library



Results:

100 epoch and loop = 20

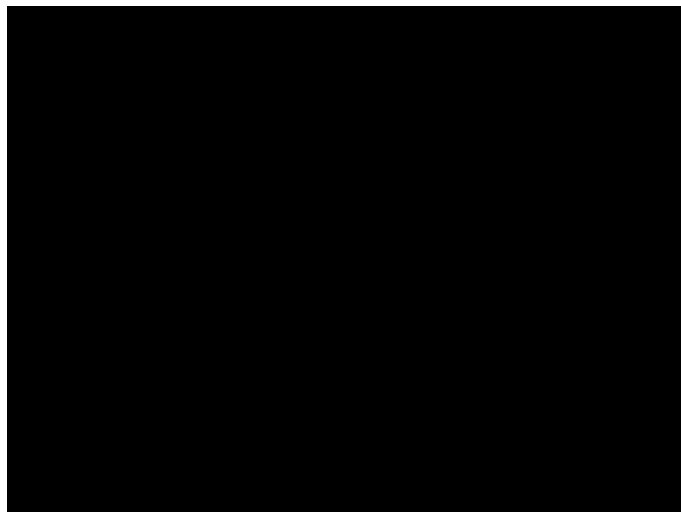


Point Transformer vs Softgroup

Assuming that Point Transformer loop = 20 is similar to repeat = 20 from Softgroup, we have:

Val\Model	Point Transformer - 100 epoch - 2.289 day	SoftGroup - 50 epoch - 22h
Acc	0.74	0.86
mIoU	0.66	0.60

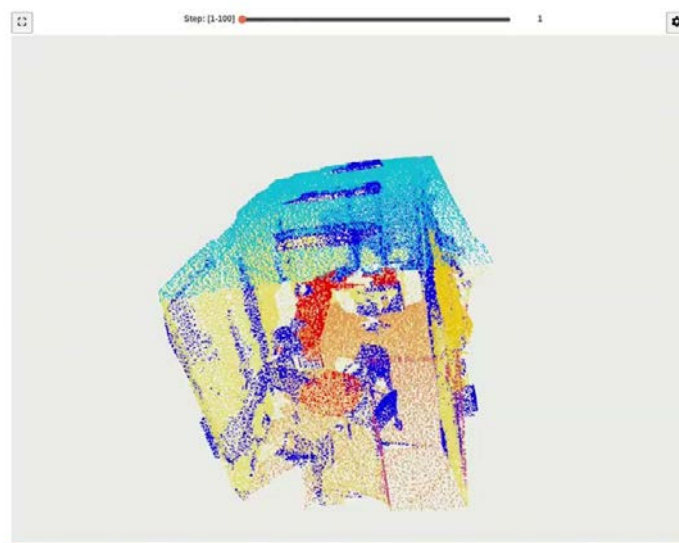
Qualitative Analysis



Qualitative Analysis



Qualitative Analysis



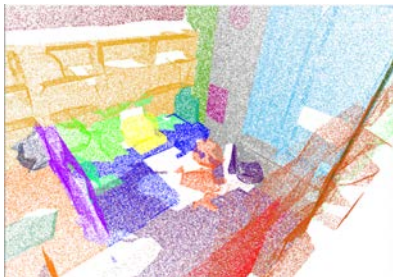
Space for research and improvements

- Evaluation in new contexts
- Adaptation for instance segmentation task
- Improve the efficiency of the kNN algorithm
- Make it faster!

Evaluation in new contexts

- S3DIS, Model40Net and ShapeNetPart are very well-behaved datasets.
- Specific use cases:

S3DIS:



ModelNet40:

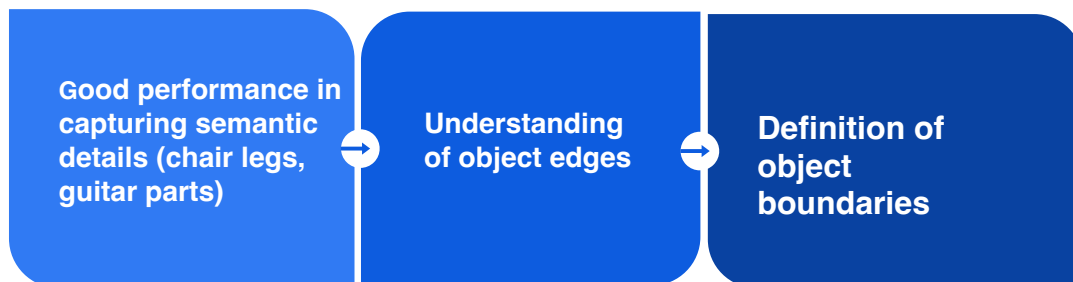


ShapeNetPart:



Adaptation for instance segmentation task

- Point Transformer only semantic segmentation and object parts
- Idea:



Improve k-NN efficiency

- Cost:
- Local Self-attention 
- kNN search 

#pts	k=8	k=16	k=32	k=64	k=128	k=256
10k	2	2	5	10	17	21
20k	3	5	8	23	43	49
40k	8	12	26	71	127	144
80k	23	37	82	198	356	399
100k	32	46	99	248	445	494
200k	104	125	225	545	992	1091
500k	639	695	867	1589	2865	3143
1m	2496	2648	2949	4087	6362	6878

Make it faster

- Inference can be improved (719 ms for 80k points).
- Goal: to be used by robots and autonomous cars.



Observation: Point Transformer v3

- The last 3 points were worked on by the authors and improved in version 3 of the Point Transformer.
- **Adaptation to instance segmentation tasks:** Point Transformer v3 proved capable of performing instance segmentation on the main datasets.
- **Improve kNN efficiency:** Use of a serialized neighbor mapping mechanism
- **Make it faster:** The inference time is 3.3x lower than its predecessor.

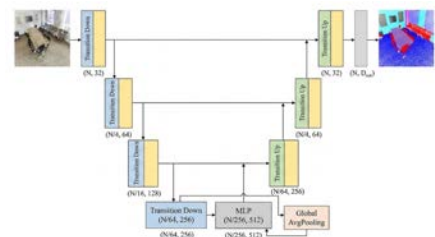
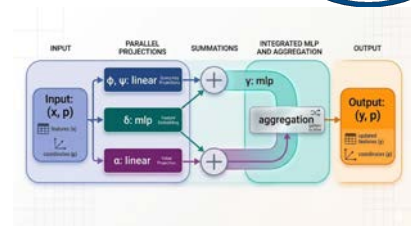
It doesn't stop there!

The review

- The paper is evaluated along five axes:
summary, strengths, weaknesses, rating, suggestions.
- Guiding principles followed:
 - weigh **novelty and impact**, not only state-of-the-art margins;
 - be **specific** about prior art;
 - keep feedback **constructive**.

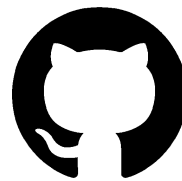
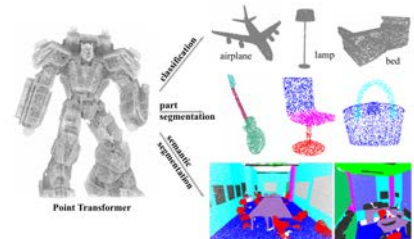
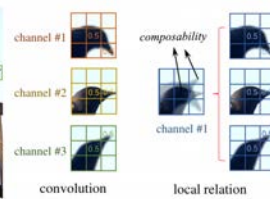
Summary

- **Point Transformer layer**, that Generates unique weight vectors for individual feature channels
- **Relative Position Encoding** Uses a trainable MLP-based encoding for relative point coordinates
- **Scalability**: Applies attention within local k -nearest neighbor (kNN) regions
- **Residual U-Net Design** Employs a symmetric encoder-decoder structure with Transition Down (FPS/Max-pooling) and Transition Up (Interpolation) modules
- **State-of-the-Art Performance** First model to cross **70% mIoU** on S3DIS Area 5

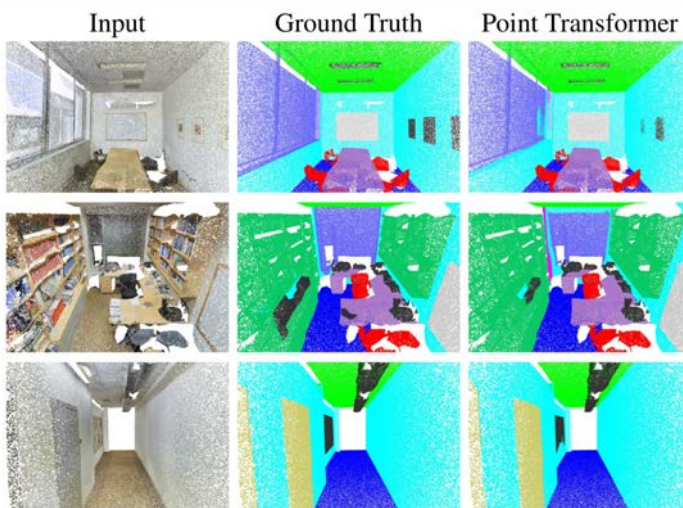


Strengths

- **Innovation (8/10)**: Adaptation of the Transformer to point clouds. Using Vector self-attention is applied **locally** (within KNN)
- **Results (9/10)**: state-of-the-art segmentation on the **S3DIS**; Classification **ModelNet40**; And part segmentation **ShapeNet-Part**.
- **Writing (10/10)**: Summary; experiment; standard metrics for comparison; datasets, code, ablations; and influential prior work properly cited.
- **Reproducibility(10/10)**: code is publicly available and now maintained within the **Pointcept** framework.



Results



Semantic **segmentation** results on the S3DIS dataset, evaluated on Area 5.

Method	OA	mAcc	mIoU
PointNet [25]	–	49.0	41.1
SegCloud [36]	–	57.4	48.9
TangentConv [35]	–	62.2	52.6
PointCNN [20]	85.9	63.9	57.3
SPGraph [15]	86.4	66.5	58.0
PCCN [42]	–	67.0	58.3
PAT [50]	–	70.8	60.1
PointWeb [55]	87.0	66.6	60.3
HPEIN [13]	87.2	68.3	61.9
MinkowskiNet [37]	–	71.7	65.4
KPConv [37]	–	72.8	67.1
PointTransformer	90.8	76.5	70.4

Weaknesses

- Improvements over the state of the art are **incremental on two of the three tasks**: $\approx +0.7\%$ OA on ModelNet40 (classification) and $\approx +0.2\%$ instance mIoU on ShapeNet-Part (part segmentation).
- **Evaluation breadth is limited**: each task is validated on essentially one standard benchmark (detailed in the closing observation).
- Per CVPR guidance, **neither** thin margins **nor** limited dataset coverage is, by itself, grounds for rejection — these are recorded as limitations, not as decisive faults.

Method	input	mAcc	OA
3DShapeNets [47]	voxel	77.3	84.7
VoxNet [23]	voxel	83.0	85.9
Subvolume [26]	voxel	86.0	89.2
MVCNN [34]	image	–	90.1
PointNet [25]	point	86.2	89.2
A-SCN [48]	point	87.6	90.0
Set Transformer [17]	point	–	90.4
PAT [50]	point	–	91.7
PointNet++ [27]	point	–	91.9
SpecGCN [40]	point	–	92.1
PointCNN [20]	point	88.1	92.2
DGCNN [44]	point	90.2	92.2
PointWeb [55]	point	89.4	92.3
SpiderCNN [49]	point	–	92.4
PointConv [46]	point	–	92.5
Point2Sequence [21]	point	90.4	92.6
KPConv [37]	point	–	92.9
InterpCNN [22]	point	–	93.0
PointTransformer	point	90.6	93.7

Table 3. Shape classification results on the ModelNet40 dataset.

Method	cat. mIoU	ins. mIoU
PointNet [25]	80.4	83.7
A-SCN [48]	–	84.6
PCNN [42]	81.8	85.1
PointNet++ [27]	81.9	85.1
DGCNN [44]	82.3	85.1
Point2Sequence [21]	–	85.2
SpiderCNN [49]	81.7	85.3
SPLATNet [33]	83.7	85.4
PointConv [46]	82.8	85.7
SGPN [43]	82.8	85.8
PointCNN [20]	84.6	86.1
InterpCNN [22]	84.0	86.3
KPConv [37]	85.1	86.4
PointTransformer	83.7	86.6

Table 4. Object part segmentation results on the ShapeNetPart dataset.

Rating & justification

Recommendation:  Accept.

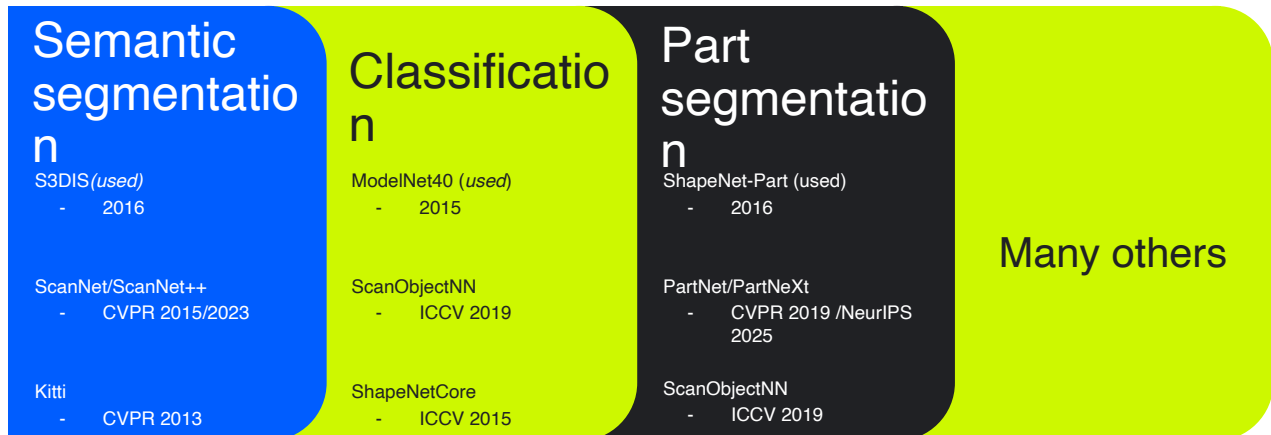
Innovation	8
Results	9
Writing	10
Reproducibility	10

Suggestions for the authors

- **Broaden the evaluation** to more challenging and varied public datasets, to demonstrate robustness more convincingly (detail on the next slide).
- **Report efficiency (latency / memory)** alongside accuracy, to make the advantage of local over global attention explicit.

Closing observation: breadth of evaluation

Demonstrating robustness usually means testing across varied contexts; with modest extra effort, additional public datasets were available for each task:



Postscript: the field's own answer (Point Transformer V3)

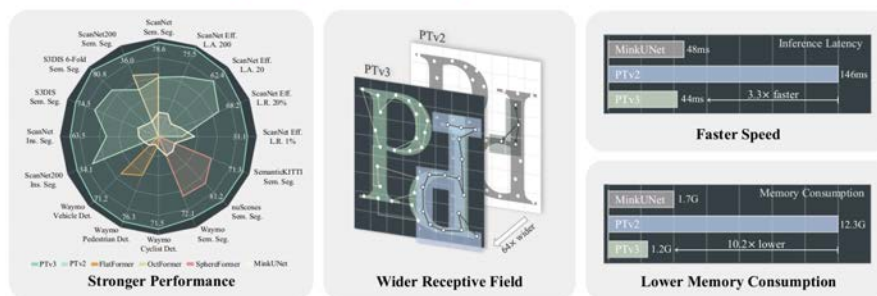
Point Transformer V3: Simpler, Faster, Stronger

Xiaoyang Wu^{1,2} Li Jiang³ Peng-Shuai Wang⁴
Zhijian Liu⁵ Xihui Liu¹ Yu Qiao² Wanli Ouyang² Tong He^{2*} Hengshuang Zhao^{1*}

¹HKU ²SH AI Lab ³CUHK(SZ) ⁴PKU ⁵MIT

<https://github.com/Pointcept/PointTransformerV3>

CVJ 25 Mar 2024



A note on space-filling serialization

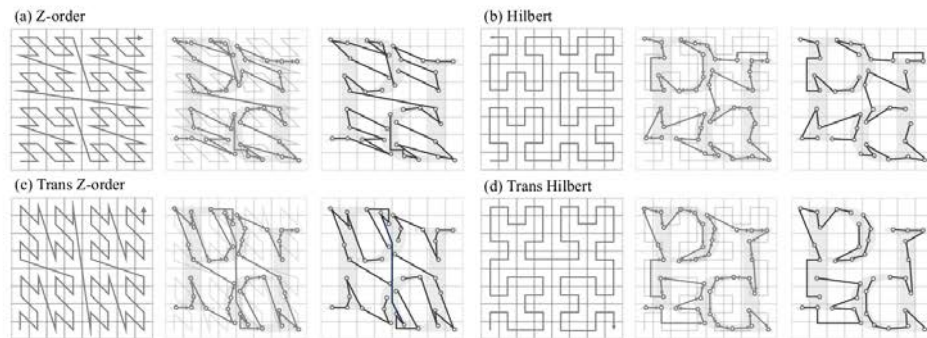


Figure 3. **Point cloud serialization.** We show the four patterns of serialization with a triplet visualization. For each triplet, we show the space-filling curve for serialization (left), point cloud serialization var sorting order within the space-filling curve (middle), and grouped patches of the serialized point cloud for local attention (right). Shifting across the four serialization patterns allows the attention mechanism to capture various spatial relationships and contexts, leading to an improvement in model accuracy and generalization capacity.



Additionally, Pointcept integrates the following excellent work (contain above):
Backbone: [MinkUNet \(here\)](#), [SpUNet \(here\)](#), [SPVCNN \(here\)](#), [OACNNs \(here\)](#), [PTv1 \(here\)](#), [PTv2 \(here\)](#), [PTv3 \(here\)](#),
[StratifiedFormer \(here\)](#), [OctFormer \(here\)](#), [Swin3D \(here\)](#), [LitePT \(here\)](#);
Semantic Segmentation: [Mix3d \(here\)](#), [CAC \(here\)](#);
Instance Segmentation: [PointGroup \(here\)](#);
Pre-training: [PointContrast \(here\)](#), [Contrastive Scene Contexts \(here\)](#), [Masked Scene Contrast \(here\)](#), [Point Prompt Training \(here\)](#), [Sonata \(here\)](#), [Concerto \(here\)](#), [Utonia \(here\)](#);
Datasets: [ScanNet \(here\)](#), [ScanNet200 \(here\)](#), [ScanNet++ \(here\)](#), [S3DIS \(here\)](#), [ArkitScene \(here\)](#), [HM3D \(here\)](#),
[Matterport3D \(here\)](#), [Structured3D \(here\)](#), [SemantickITTI \(here\)](#), [nuScenes \(here\)](#), [Waymo \(here\)](#), [ModelNet40 \(here\)](#),
[ScanObjectNN \(here\)](#), [ShapeNetPart \(here\)](#), [PartNetE \(here\)](#).

About:

“[Pointcept](#): Perceive the world with sparse points, a codebase for point cloud perception research. Latest works: Utonia (**ICML'26**), Concerto (**NeurIPS'25**), Sonata (**CVPR'25 Highlight**), PTv3 (**CVPR'24 Oral**)”